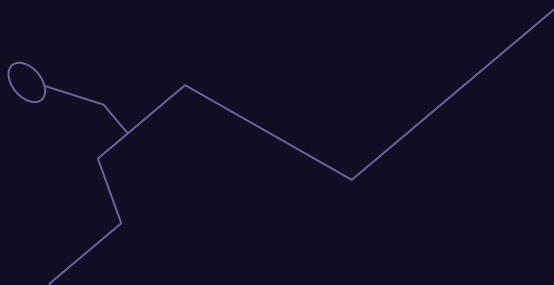


Nền tảng phát triển web

Lập trình hướng đối tượng (OOP) với JS



▶ Lập trình OOP với JS

01

Giới thiệu sơ lược về JS

02

Sơ lược về OOP

03

Các đặc trưng của OOP trong JS

04

Ví dụ

▶ Giới thiệu về Javascript

01

1. Giới thiệu về Javascript

JS là gì?

- JavaScript là một trong những ngôn ngữ lập trình quan trọng nhất trong lĩnh vực phát triển web và ứng dụng. Nó cung cấp các công cụ và khả năng mạnh mẽ để tạo ra các trang web tương tác và ứng dụng web đa dạng.
- JS ban đầu được tạo ra để cung cấp các tính năng tương tác và động cho các trang web, nhưng ngày nay nó đã mở rộng để sử dụng trong nhiều lĩnh vực khác nhau, bao gồm phát triển ứng dụng di động và desktop.

JavaScript



► Sơ lược về OOP

02

2. Sơ lược về OOP

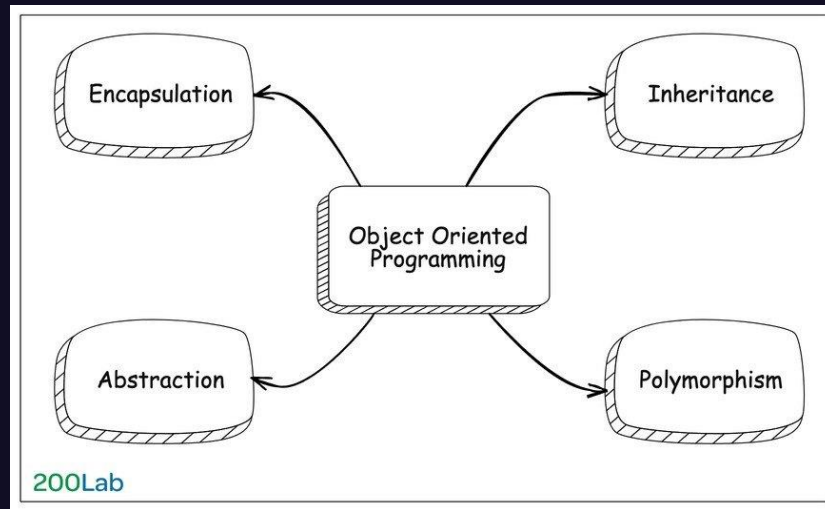
- OOP là viết tắt của Object-oriented programming (lập trình hướng đối tượng). OOP là một mô hình lập trình dựa trên khái niệm Object (đối tượng), mà trong đó thường chứa 2 thành phần: data và code.
- Data thường dưới dạng các fields, hay còn gọi là các attributes hoặc properties (tạm dịch chung là các thuộc tính).
- Code thường dưới dạng các procedures, hay còn gọi là methods (tạm dịch là các phương thức).

O **OBJECT**
O **ORIENTED**
P **PROGRAMMING**

2. Sơ lược về OOP

Các đặc trưng cơ bản của OOP

- **Encapsulation** (tính đóng gói)
- **Inheritance** (Tính kế thừa)
- **Polymorphism** (Tính đa hình)
- **Abstraction** (Tính trừu tượng)



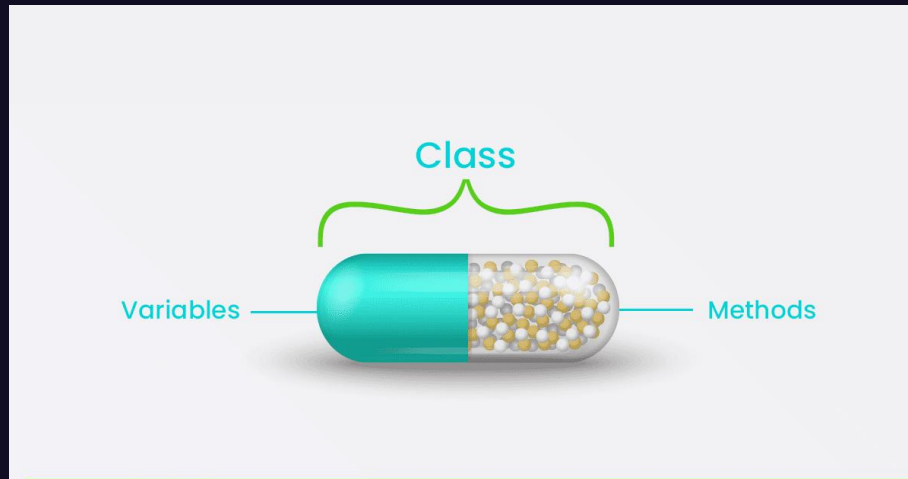
► Các đặc trưng của OOP trong JS

03

▶ 3.1 ENCAPSULATION (TÍNH ĐÓNG GÓI)

▶ *Tính đóng gói là gì?*

- Tính đóng gói là kỹ thuật giúp bạn che giấu được những thông tin bên trong đối tượng.
- Mục đích chính của tính đóng gói là giúp hạn chế các lỗi khi phát triển chương trình.
- Tính chất này không cho phép người sử dụng các đối tượng thay đổi trạng thái nội tại của một đối tượng. Chỉ có các phương thức nội tại của đối tượng cho phép thay đổi trạng thái của nó.
- Việc cho phép môi trường bên ngoài tác động lên các dữ liệu nội tại của một đối tượng theo cách nào là hoàn toàn tùy thuộc vào người viết mã.
=> Đây là tính chất đảm bảo sự toàn vẹn của đối tượng.



▶ 3.1 ENCAPSULATION (TÍNH ĐÓNG GÓI)

▶ *Các lợi ích chính của Encapsulation*

- Hạn chế được các truy xuất không hợp lệ tới các thuộc tính của đối tượng.
- Giúp cho trạng thái của đối tượng luôn đúng.
- Giúp ẩn đi những thông tin không cần thiết về đối tượng.
- Cho phép bạn thay đổi cấu trúc bên trong lớp mà không ảnh hưởng tới lớp khác.

▶ *Cách thực hiện Encapsulation trong JS*

- Trong **Javascript**, để thực hiện tính đóng gói, ta có thể tạo ra 1 *Constructor Function*, đóng gói toàn bộ các trường và hàm vào 1 object.
- Để đóng gói trong JS, ta dùng
 - từ khóa “var” để đặt dữ liệu ở chế độ riêng tư.
 - các phương thức setter để đặt dữ liệu và các phương thức getter để lấy dữ liệu đó.
- Việc đóng gói cho phép chúng ta xử lý một đối tượng bằng các thuộc tính sau:
 - Đọc/Ghi - Ở đây, chúng ta sử dụng các phương thức setter để ghi dữ liệu và các phương thức getter đọc dữ liệu đó.
 - Chỉ đọc - Trong trường hợp này, chúng ta chỉ sử dụng các phương thức getter.
 - Chỉ ghi - Trong trường hợp này, chúng ta chỉ sử dụng các phương thức setter

▶ 3.1 ENCAPSULATION (TÍNH ĐÓNG GÓI)

▶ Ví dụ minh họa Encapsulation

Không sử dụng Encapsulation

```
<script>
    function Person(firstName, lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.showName = function() {
            console.log(this.firstName + ' ' + this.lastName);
        };
    }

    var psn1 = new Person('Biden', 'Vladimir');

    // các property khai báo vào biến this có thể bị truy xuất từ bên ngoài
    // object không còn bao đóng nữa
    psn1.firstName = 'changed';
    console.log(psn1.firstName); // changed
</script>
```

▶ 3.1 ENCAPSULATION (TÍNH ĐÓNG GÓI)

▶ Ví dụ minh họa Encapsulation

Sử dụng Encapsulation

```
<script>
function Person(firstName, LastName) {
    var fstName = firstName;
    var lstName = LastName;

    this.setFirstName = function(firstName) {
        fstName = firstName;
    };

    this.getFirstName = function() {
        return fstName;
    };

    this.setLastName = function(LastName) {
        lstName = LastName;
    };

    this.getLastName = function() {
        return lstName;
    };
}

var person1 = new Person('Biden', 'Vladimir');
console.log(person1.fstName); // Undefined, không thể truy cập được

console.log(person1.getFirstName());
</script>
```

▶ 3.2 Inheritance (Tính kế thừa)

- ▶ *Tính kế thừa là gì?*
- ▶ *Class Inheritance trong JS là gì?*
- ▶ *Phương thức Super trong JS*
- ▶ *Ghi đè phương thức hoặc thuộc tính*

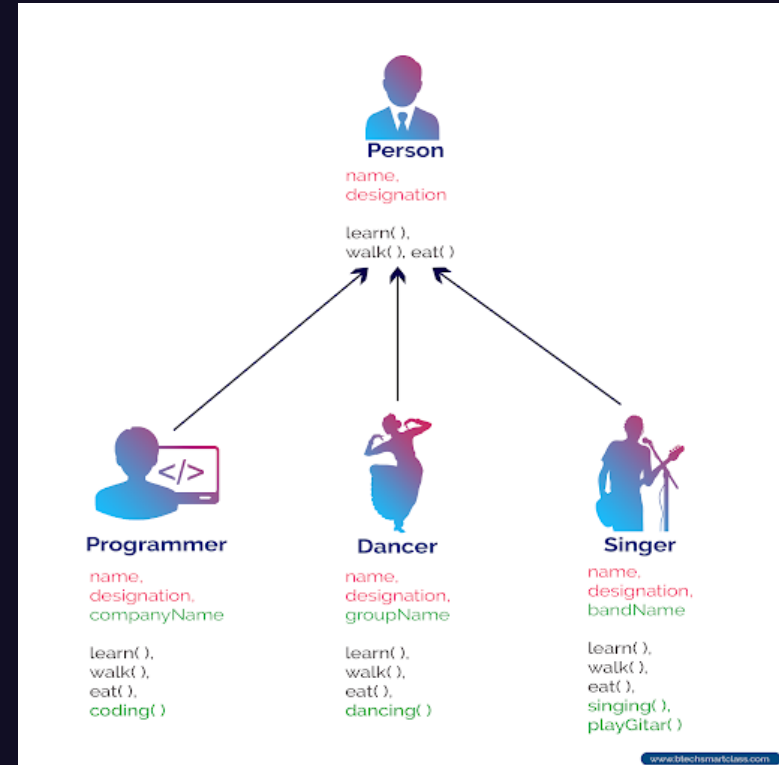
3.2 Inheritance (Tính kế thừa)

Tính kế thừa là gì?

- Kế thừa trong lập trình hướng đối tượng chính là thừa hưởng lại những thuộc tính và phương thức của một lớp. Có nghĩa là nếu lớp A kế thừa lớp B thì lớp A sẽ có những thuộc tính và phương thức của lớp B.
- Lớp được thừa hưởng những thuộc tính và phương thức từ lớp khác được gọi là dẫn xuất (*Derived Class*) hay lớp Con (*Subclass*) và lớp bị lớp khác kế thừa được gọi là lớp cơ sở (*Base Class*) hoặc lớp cha (*Parent Class*).

Các lợi ích của Inheritance

- Giúp tái sử dụng lại code.
- Tăng khả năng mở rộng của chương trình.



3.2 Inheritance (Tính kế thừa)

Class Inheritance trong JS là gì?

- Class Inheritance trong Js chính là tính kế thừa của lớp trong Js. Nó cho phép một lớp có thể thừa kế tất cả các hàm từ một lớp cha và cho phép định nghĩa thêm các hàm khác
- Bằng cách tận dụng tính kế thừa lớp trong Js, một lớp có thể kế thừa tất cả các phương thức và thuộc tính của lớp khác.
- Để thực hiện việc kế thừa từ một lớp khác, ta sẽ sử dụng từ khóa "extends"

```
<script>
class Car {
  constructor(brand) {
    this.carname = brand;
  }
  present() {
    return 'Tôi có một chiếc xe hãng ' + this.carname;
  }
}
class Model extends Car {
  constructor(brand, mod) {
    super(brand);
    this.model = mod;
  }
  show() {
    return this.present() + ', Nó là một chiếc ' + this.model;
  }
}
let myCar = new Model("Ford", "Mustang");
document.write(myCar.show());
</script>
```

3.2 Inheritance (Tính kế thừa)

Phương thức Super trong JS

- Phương thức `super()` tham chiếu đến lớp cha. Tức là nó được sử dụng bên trong lớp con để biểu thị lớp cha của nó (Lớp mà nó kế thừa).
- Bằng cách gọi phương thức `super()` trong phương thức khởi tạo, chúng ta gọi phương thức khởi tạo của cha và nhận được quyền truy cập vào các thuộc tính và phương thức của cha.

```
<script>
  class sinh_vien {
    constructor(a) {
      this.ten = a;
    }
    in_thong_tin = function () {
      console.log(this.ten);
    }
  }
  class sv_Luat extends sinh_vien {
    constructor(b) {
      document.write("Gọi hàm cha");
      super(b);
    }
  }
  let sv = new sv_Luat('Nam');
  sv.in_thong_tin();
</script>
```


3.2 Inheritance (Tính kế thừa)

Ghi đè phương thức hoặc thuộc tính

Nếu một lớp con có cùng tên của phương thức hoặc thuộc tính với tên của phương thức hoặc thuộc tính trong lớp cha, nó sẽ sử dụng phương thức và thuộc tính của lớp con. Ta gọi nó là ghi đè phương thức hoặc thuộc tính

Ở đây, thuộc tính ID và phương thức `in_thong_tin()` có tồn tại trong lớp cha `sinh_vien` và lớp con `sv_IT`. Như vậy, lớp `sv_IT` sẽ ghi đè thuộc tính ID và phương thức `in_thong_tin()` trong lớp cha

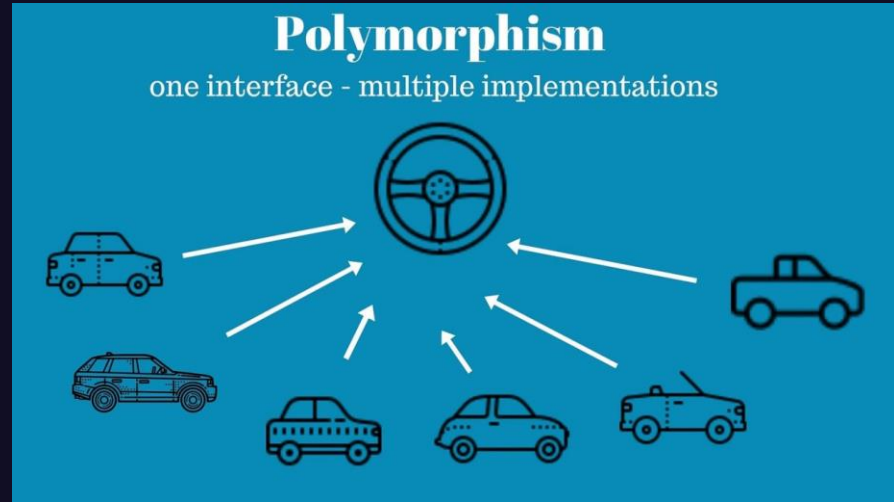
```
<script>
class sinh_vien {
  constructor(a) {
    this.ten = a;
    this.ID = 100;
  }
  in_thong_tin = function () {
    document.write(this.ten);
  }
}

class sv_IT extends sinh_vien {
  constructor(b) {
    document.write("Gọi hàm cha -");
    super(b);
    this.ID = 230;
  }
  in_thong_tin = function () {
    document.write(this.ten + ' - ');
    document.write(this.ID);
    document.write("- Ghi đè hàm.");
  }
}

let sv = new sv_IT('Ahyhy');
sv.in_thong_tin();
</script>
```

3.3 POLYMORPHISM (TÍNH ĐA HÌNH)

- Tính đa hình thể hiện thông qua việc gửi các thông điệp (message). Việc gửi các thông điệp này có thể so sánh như việc gọi các hàm bên trong của một đối tượng.
- Các phương thức dùng trả lời cho một thông điệp sẽ tùy theo đối tượng mà thông điệp đó được gửi tới sẽ có phản ứng khác nhau.
- Người lập trình có thể định nghĩa một đặc tính (chẳng hạn thông qua tên của các phương thức) cho một loạt các đối tượng gần nhau nhưng khi thi hành thì dùng cùng một tên gọi mà sự thi hành của mỗi đối tượng sẽ tự động xảy ra tương ứng theo đặc tính của từng đối tượng mà không bị nhầm lẫn.



=> Tính đa hình cung cấp cách thức thực hiện một hành động dưới các hình thức khác nhau.

Trong JS, Nó cung cấp khả năng gọi cùng một phương thức trên các đối tượng khác nhau.

3.3 POLYMORPHISM (TÍNH ĐA HÌNH)

```
<script>
  class Language {}

  class English extends Language {
    greeting() {
      console.log("Hello");
    }
  }

  class French extends Language {
    greeting() {
      console.log("Bonjour");
    }
  }

  function intro(language) {
    // Check type of 'language' object.
    if(language instanceof Language) {
      language.greeting();
    }
  }

  // ----- TEST -----
  let flora = new English();
  let aalase = new French();

  // Call function:
  intro(flora);
  intro(aalase);

  let someObject = {};

  intro(someObject);
</script>
```

Trong ví dụ này, ta tạo hai đối tượng flora và aalase từ hai lớp con khác nhau của Language và một đối tượng someObject thông thường.

Sau đó, ta gọi hàm intro với mỗi đối tượng và kiểm tra xem nó có thực sự là một thể hiện của Language không trước khi gọi phương thức greeting().

Tính đa hình được thể hiện như thế nào?

- Phương thức greeting() được định nghĩa trong cả hai class con English và French, nhưng cách thực hiện của chúng khác nhau.
- Hàm intro(language) sử dụng đa hình khi kiểm tra xem đối tượng được truyền vào có phải là một instance của Language hay không. Việc này giúp giữ cho mã nguồn linh hoạt, có thể xử lý các đối tượng thuộc các lớp con của Language mà không cần biết chính xác là đối tượng nào.

3.3 POLYMORPHISM (TÍNH ĐA HÌNH)

Tính đa hình được thể hiện như thế nào?

```
<script>
class DongVat {
    keu() {
        console.log('Động vật kêu');
    }
}

// Lớp chó kế thừa từ lớp động vật
class Cho extends DongVat {
    keu() {
        console.log('Gâu gâu!');
    }
}

// Lớp mèo kế thừa từ lớp động vật
class Meo extends DongVat {
    keu() {
        console.log('Meo meo!');
    }
}

// Sử dụng tính đa hình
const dongVat = new DongVat();
dongVat.keu(); // Kết quả: Động vật kêu

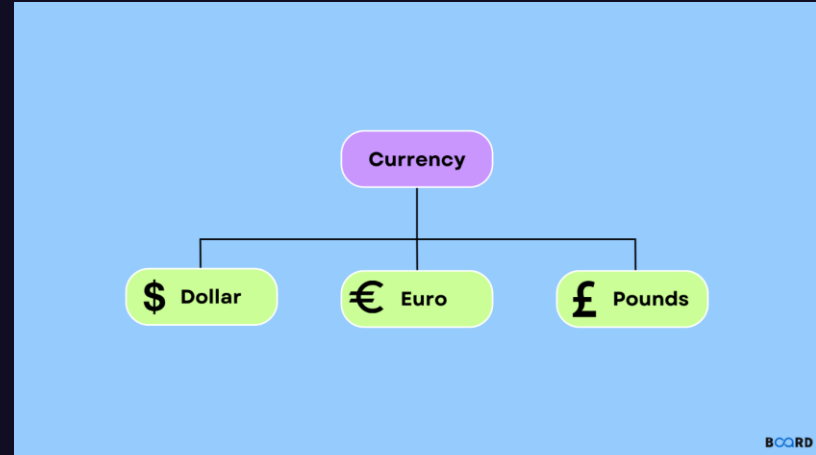
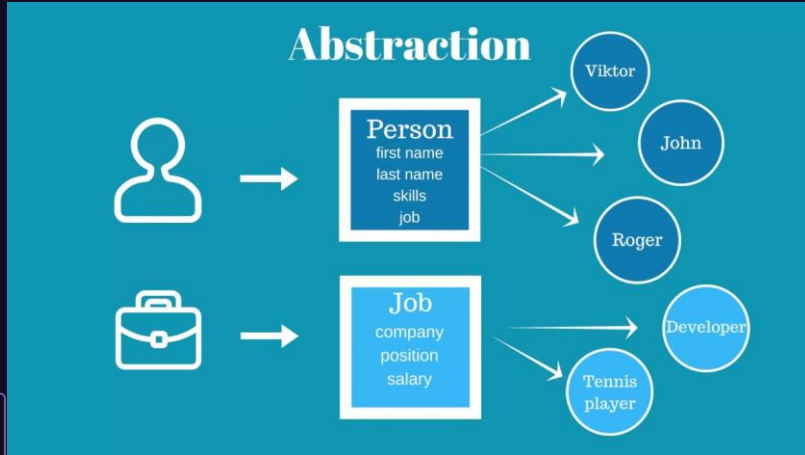
const cho = new Cho();
cho.keu(); // Kết quả: Gâu gâu!

const meo = new Meo();
meo.keu(); // Kết quả: Meo meo!
</script>
```

- Trong ví dụ này, lớp DongVat định nghĩa một phương thức keu() mà các lớp con có thể triển khai lại theo cách riêng của chúng.
- Lớp Cho và Meo kế thừa từ lớp DongVat và triển khai lại phương thức keu() theo cách riêng của chúng. Điều này cho phép các đối tượng được tạo từ các lớp con gọi phương thức keu() tương ứng với lớp con đó.
- Khi chúng ta tạo các đối tượng và gọi phương thức keu(), chúng ta nhận được kết quả phù hợp với từng lớp con, mặc dù chúng đều được gọi thông qua cùng một phương thức trong lớp cha.

3.4 ABSTRACTION (TÍNH TRỪU TƯỢNG)

Tính trừu tượng là một tính chất mà chỉ tập trung vào những tính năng của đối tượng và ẩn đi những thông tin không cần thiết. Tính chất này giúp bạn trọng tâm hơn vào những tính năng thay vì phải quan tâm tới cách mà nó được thực hiện.



Trừu tượng trong JavaScript là một khái niệm trong lập trình hướng đối tượng, nơi chúng ta tạo ra các lớp (class) và định nghĩa các thuộc tính và phương thức mà lớp đó có. Trừu tượng cho phép chúng ta tạo ra các lớp mà không cần triển khai chi tiết cụ thể của chúng, mà chỉ định nghĩa các hành vi chung mà các lớp đó nên có.

3.4 ABSTRACTION (TÍNH TRỪU TƯỢNG)

```
<script>
// Lớp hình học trừu tượng
class HìnhHoc {
    tínhDienTich() {
        throw new Error('Phương thức chưa được định nghĩa');
    }
    tínhChuVi() {
        throw new Error('Phương thức chưa được định nghĩa');
    }
}

// Lớp hình vuông kế thừa từ lớp hình học
class HìnhVuong extends HìnhHoc {
    constructor(canh) {
        super();
        this.canh = canh;
    }
    tínhDienTich() {
        return this.canh * this.canh;
    }
    tínhChuVi() {
        return 4 * this.canh;
    }
}

// Lớp hình tròn kế thừa từ lớp hình học
class HìnhTron extends HìnhHoc {
    constructor(banKinh) {
        super();
        this.banKinh = banKinh;
    }
    tínhDienTich() {
        return Math.PI * this.banKinh * this.banKinh;
    }
    tínhChuVi() {
        return 2 * Math.PI * this.banKinh;
    }
}

// Sử dụng lớp hình vuông
const hìnhVuong = new HìnhVuong(5);
console.log('Diện tích hình vuông:', hìnhVuong.tínhDienTich()); // Kết quả: 25
console.log('Chu vi hình vuông:', hìnhVuong.tínhChuVi()); // Kết quả: 20

// Sử dụng lớp hình tròn
const hìnhTron = new HìnhTron(3);
console.log('Diện tích hình tròn:', hìnhTron.tínhDienTich()); // Kết quả: 28.274333882308138
console.log('Chu vi hình tròn:', hìnhTron.tínhChuVi()); // Kết quả: 18.84955592153876
</script>
```

Trong ví dụ này, lớp HìnhHoc định nghĩa hai phương thức tínhDienTich và tínhChuVi, nhưng chúng chỉ ném ra một lỗi khi được gọi. Điều này đảm bảo rằng các lớp con phải định nghĩa lại các phương thức này.

Lớp HìnhVuong và HìnhTron kế thừa từ lớp HìnhHoc và triển khai lại các phương thức tínhDienTich và tínhChuVi theo cách riêng của chúng.

Khi chúng ta tạo các đối tượng từ các lớp con và gọi các phương thức tương ứng, chúng ta nhận được kết quả phù hợp với từng lớp.

So sánh POLYMORPHISM và ABSTRACTION



	Polymorphism	Abstraction
Mục tiêu	Tính đa hình là khả năng của các đối tượng cung cấp cùng một giao diện chung (phương thức hay thuộc tính) mà có thể được triển khai khác nhau tùy thuộc vào lớp cụ thể của đối tượng đó.	Tính trừu tượng là việc tạo ra các lớp cơ sở (base classes) mô tả các khái niệm tổng quát, với các phương thức và thuộc tính mà các lớp con có thể kế thừa
Ưu điểm	Cho phép sử dụng một giao diện chung để tương tác với các đối tượng khác nhau, giảm sự phụ thuộc vào cài đặt cụ thể.	Giúp ẩn đi các chi tiết cụ thể, tập trung vào các khái niệm chính, giúp dễ dàng duy trì và mở rộng chương trình.

▶ Chương trình minh họa

04